Atividade avaliativa

- Considere os 4 tópicos abaixo para envios da atividade.
- A composição AT1 e AT2 será:
  AT1 = `(item 2 + item 3) * 0.4`
  AT2 = `(item 1 + item 4) * 0.5`
- O peso de cada item: item 1, item 2, item 3 e item 4, valendo 1,2,3,4 respectivamente.
- Os item 1, 2 e 3 deverão ser enviados em um arquivo **grupo_N_docs.pdf**, sendo N o número do grupo na plataforma AVA.
- O item 4 deverá ser enviado como um projeto Java como **grupo_N_projeto.zip,** contendo todo o código do projeto
- Para o item 4, sempre o exercício seguinte substitui/refatora o anterior; *não são 20 programas independentes*.
- A previsão é que os GRUPOS, não consumam mais do que 1 hora para cada item de 1 até o 3 e não mais que 3 horas para o item 4 (considerando uma estimativa individual para uma atividade em grupo).
- Todos os integrantes dos grupos deverão estar cadastrados no grupo correto na plataforma AVA, caso algum integrante não esteja formalmente cadastrado no grupo até a entrega da atividade, o mesmo deverá realizar a avaliação N3 com teor substitutivo no final do semestre.
- Caso sejam identificadas cópias de trechos ou integral entre os itens/grupos não será atribuída nota a nenhum dos grupos para o item específico.
- Não haverá dilatação de prazo para envio da atividade.

1 - Business Model Canvas

- Preencham um BMC do projeto proposto pelo grupo, 9 blocos com respostas pragmáticas, lembrando que é um documento extremamente subjetivo e será nossa primeira versão, então preencham da melhor forma (avaliação quantitativa) que conseguirem que vamos melhorar durante as próximas aulas.
- Exemplo anexo na atividade

2 - Telas

- Identifiquem os usuários do sistema (Síndico, Porteiro, Chefe de manutenção, Moradores, Visitantes ...), elaborem as primeiras telas (wireframe) no Figma, baixa fidelidade.
- Compartilhem o link (público do Figma, testem antes de enviar, deverei conseguir acessar sem autenticação e aba anônima).

3 - Pesquisa bibliográfica

- Façam uma pesquisa simples (preferencialmente nas referências disponibilizadas no plano de ensino), referente aos **pilares da POO**, apresentem as definições de cada um dos conceitos pelo ponto de vista do(s) autor(es), mesmo que seja no estilo de citação literal).
- Discutam em grupo e caso algum conceito não fique claro conforme apresentado pelo autor, tragam também as dúvidas para nossas próximas aulas.

4 - Lista de exercícios

1. **Criando o projeto e identificando o condomínio**
   Crie o projeto Java `gestao-condominio` e a classe `App`. No `main`, declare variáveis para representar o nome do condomínio, quantidade de blocos, quantidade de unidades, valor da taxa condominial e se o condomínio possui portaria 24 horas. Exiba uma ficha como:

```
===== CONDOMÍNIO =====
Nome: Residencial Java Garden
Blocos: 2
Unidades: 24
Taxa padrão: R$ 550.00
Portaria 24h: true
```

Objetivo: tipos primitivos, `String`, variáveis e `System.out.println()`.

2. **Lendo os dados do condomínio pelo terminal**
   Substitua os valores fixos do exercício anterior por dados informados pelo usuário utilizando `Scanner`. Solicite nome, quantidade de blocos, quantidade de unidades e taxa padrão.
   Objetivo: `Scanner`, entrada e saída de dados.
3. **Classificando a taxa condominial**
   Após ler a taxa, informe:

```
Até R$ 400      -> Taxa econômica
R$ 400,01 a 700 -> Taxa intermediária
Acima de R$ 700 -> Taxa elevada
```

Objetivo: `if`, `else if` e `else`.

4. **Simulando arrecadação mensal**
   Solicite a quantidade de unidades e o valor da taxa. Calcule quanto o condomínio arrecadaria caso todas as unidades pagassem. Em seguida, utilizando `for`, exiba uma linha para cada unidade:

```
Unidade 1 - Taxa: R$ 550.00
Unidade 2 - Taxa: R$ 550.00
...
```

Objetivo: `for`, cálculos e repetição.

5. **Criando o menu do sistema**
   Crie um menu executado continuamente:

```
1 - Dados do condomínio
2 - Simular arrecadação
0 - Sair
```

O programa deverá continuar executando até o usuário escolher `0`.
Objetivo: `while`, decisões e controle de fluxo.

6. **Criando a primeira classe do domínio:** `Morador`
   Crie a classe:

```
Morador
```

com os atributos:

```
String nome;
String cpf;
int numeroUnidade;
```

No `App`, crie dois moradores utilizando `new Morador()` e atribua valores aos atributos. Depois exiba os dados.
Objetivo: classe, objeto, instância e abstração.

7. **Adicionando construtor a** `Morador`
   Refatore a classe para possuir:

```
Morador(String nome, String cpf, int numeroUnidade)
```

Agora os moradores deverão ser criados assim:

```
Morador morador1 =
   new Morador("Carlos", "12345678900", 101);
```

Objetivo: construtores e `this`.

8. **Criando a classe `Unidade`**
   Crie:

```
Unidade
```

com:

```
int numero;
double taxaBase;
boolean ocupada;
```

e um construtor. Crie no `App`:

```
Unidade 101
Unidade 102
Unidade 201
```

Exiba as informações de cada uma.
Objetivo: abstração, múltiplas classes e múltiplos objetos.

9. **Criando comportamento em `Unidade`**
   Acrescente:

```
double calcularTaxa()
```

que retorna o valor da taxa condominial da unidade. Acrescente também:

```
void exibirSituacao()
```

que mostre se a unidade está ocupada ou desocupada.
Objetivo: métodos e responsabilidade do objeto.

10. **Aplicando encapsulamento em `Unidade`**
    Transforme os atributos em `private`:

```
private int numero;
private double taxaBase;
private boolean ocupada;
```

Crie getters para consultar os dados. A taxa não poderá ser negativa. A validação deverá acontecer dentro da própria classe.
Objetivo: encapsulamento, `private`, getters e regra de negócio.

11. **Criando a classe `Condominio` e controlando o caixa**
    Crie:

```
Condominio
```

com:

```
private String nome;
private double saldoCaixa;
```

Implemente:

```
public void receberPagamento(double valor)
public void pagarDespesa(double valor)
public double getSaldoCaixa()
```

Regras:

- pagamentos devem ser positivos;
- despesas devem ser positivas;
- não permitir uma despesa superior ao saldo disponível.

Objetivo: encapsulamento e regras de negócio.

12. **Registrando pagamento de condomínio**
    Integre `Unidade` e `Condominio` pelo fluxo da aplicação. O usuário escolhe uma unidade e o programa calcula sua taxa. Depois o valor é enviado ao condomínio:

```
double valor = unidade101.calcularTaxa();
condominio.receberPagamento(valor);
```

Exiba:

```
Unidade: 101
Valor pago: R$ 550.00
Saldo do condomínio: R$ 2550.00
```

Objetivo: objetos colaborando entre si sem introduzir estruturas avançadas.

13. **Sobrecarga de construtores em `Unidade`**
    Crie duas formas de construir uma unidade:

```
public Unidade(int numero, double taxaBase)
public Unidade(int numero, double taxaBase, boolean ocupada)
```

O primeiro construtor deverá utilizar:

```
this(...)
```

para chamar o segundo. Por padrão, uma unidade criada sem informar `ocupada` deverá ser considerada ocupada.
Objetivo: sobrecarga de construtores e `this(...)`.

14. **Sobrecarga do cálculo da taxa**
    Mantenha:

```
public double calcularTaxa()
```

e acrescente:

```
public double calcularTaxa(double percentualMulta)
```

Exemplo:

```
Taxa normal: R$ 500.00
Multa: 10%
Total: R$ 550.00
```

O cálculo da taxa normal não deverá ser duplicado.
Objetivo: sobrecarga de métodos e reaproveitamento de código.

15. **Criando a especialização `Cobertura`**
    Crie:

```
public class Cobertura extends Unidade
```

Toda cobertura é uma unidade, mas possui um adicional de condomínio de 25%. O construtor deverá utilizar:

```
super(...)
```

Objetivo: herança, `extends` e `super(...)`.

16. **Sobrescrevendo `calcularTaxa()` em `Cobertura`**
    A cobertura deverá redefinir:

```
@Override
public double calcularTaxa()
```

usando:

```
super.calcularTaxa()
```

e adicionando 25%.

Exemplo:

```
Unidade comum:
R$ 500.00

Cobertura:
R$ 625.00
```

Objetivo: sobrescrita, `@Override` e `super.metodo()`.

17. **Criando funcionários do condomínio**
    Crie:

```
Funcionario
```

com:

```
private String nome;
protected double salarioBase;
```

e:

```
public double calcularSalario()
```

Depois crie:

```
Zelador extends Funcionario
```

O construtor de `Zelador` deverá chamar o construtor da classe mãe.
Objetivo: segunda aplicação independente de herança.

18. **Especializando o salário do zelador**
    O `Zelador` receberá adicional de 10%. Sobrescreva:

```
@Override
public double calcularSalario()
```

reutilizando:

```
super.calcularSalario()
```

Exemplo:

```
Funcionário comum:
R$ 3000.00

Zelador:
R$ 3300.00
```

Objetivo: consolidar sobrescrita e evitar que o estudante apenas memorize o caso `Unidade/Cobertura`.

19. **Sobrescrevendo `toString()` nas classes**
    Sobrescreva `toString()` em pelo menos:

```
Morador
Unidade
Cobertura
Funcionario
Zelador
```

O código:

```
System.out.println(unidade101);
```

poderá produzir:

```
Unidade 101 | Taxa: R$ 500.00 | Ocupada
```

enquanto:

```
System.out.println(cobertura201);
```

poderá produzir:

```
Cobertura 201 | Taxa: R$ 625.00 | Ocupada
```

Objetivo: sobrescrita de `Object.toString()` e representação textual dos objetos.

20. **Integrando o Sistema de Gestão de Condomínio**
    Reúna tudo em um menu final:

```
====================================
    RESIDENCIAL JAVA GARDEN
====================================

1 - Consultar condomínio
2 - Consultar unidade
3 - Consultar morador
4 - Calcular taxa de unidade
5 - Calcular taxa com multa
6 - Registrar pagamento
7 - Registrar despesa
8 - Consultar funcionário
9 - Simular arrecadação
0 - Sair
```

Sempre utilize alguns objetos previamente criados:

```
Unidade unidade101 = new Unidade(101, 500);
Unidade unidade102 = new Unidade(102, 500);
Cobertura cobertura201 = new Cobertura(201, 500);

Morador morador101 = new Morador("Carlos", "11111111111", 101);

Funcionario funcionario = new Funcionario("Paulo", 2500);

Zelador zelador = new Zelador("Marcos", 3000);
```

O menu deve utilizar `while` ou `do..while`, as opções podem ser controladas com `if/else if`, e todas as regras de negócio devem permanecer nas respectivas classes. O `App` deverá principalmente **ler dados, selecionar operações e chamar métodos**, evitando transformar o `main` em uma classe que faz tudo.